



Data Quality of Graph Databases

M1 Computer Science — Research Report
Data Quality of Graph Databases

Authors: Loïc DESMARÈS and Tianyi YANG
Supervisors: Béatrice FINANCE and Zoubida KEDAD

Université Paris-Saclay — UVSQ
Academic Year 2025–2026

Table of Contents

1 - Introduction	3
2 - Data Quality of a Property Graph	4
2.1 - Completeness	4
2.1.1 - Component Existence	4
2.1.2 - Node Degree	4
2.2 - Validity	4
2.2.1 - String Format	4
2.2.2 - Date Format	4
2.2.3 - Finite Data Domain	4
2.2.4 - Set-Based Labeling	5
2.2.5 - Clustering-Based Labeling	5
2.3 - Consistency	8
2.3.1 - Functional Dependency (FD)	8
2.3.2 - Conditional Functional Dependency (CFD)	8
2.3.3 - Graph Pattern Functional Dependency (GFD)	8
2.3.4 - Query-Based Validation	8
2.4 - Integrity	9
2.4.1 - Property Schema Validity	9
2.4.2 - Index Validity	9
2.4.3 - Normal Form of a Property Graph	9
2.5 - Uniqueness	10
2.5.1 - Edge Duplicates	10
2.5.2 - Node Duplicates	10
3 - Profiling of a Property Graph	11
3.1 - Completeness	11
3.1.1 - Weakly Connected Components	11
3.1.2 - Strongly Connected Components	11
3.2 - Validity	11
3.2.1 - Detection of Inconsistent Property Types	11
3.3 - Integrity	11
3.3.1 - Node Property Distribution	11
3.3.2 - Node Property Distribution by Label	11
3.3.3 - Edge Property Distribution	11
3.4 - Labeling	11
3.4.1 - Anomaly Detection via Clustering	11
3.5 - Readability	12
3.5.1 - Node Degree Distribution	12
3.5.2 - Multi-edge Detection	12
3.5.3 - Graph Eccentricity Analysis	12
3.6 - Outliers	12
3.6.1 - Numerical Outlier Detection	12
3.6.2 - Transitive Node Influence Analysis	12
3.6.3 - Average Transitive Influence Analysis	12
4 - Implementation — Neo4j	13
4.1 - Testing Methods	13
4.2 - Test Results	13
4.2.1 - Northwind	13
4.2.2 - YAGO3	14
5 - Conclusion	14
6 - Appendix	15
Bibliography	30

Abstract: The rise of digital technologies and the proliferation of heterogeneous data sources have led to an increasing adoption of graph-oriented databases, and more specifically the property graph model. This model is distinguished by its semantic richness and structural flexibility: by enabling each node and each edge to be associated with a set of labels and properties, it provides a natural and expressive representation of entities and their relationships across various domains such as social networks, bioinformatics, large-scale knowledge management, and artificial intelligence systems.

Although the state of the art and standardization of property graphs has evolved considerably, data quality aspects remain largely unexplored. Yet, as with Big Data environments, the value extracted from a graph database fundamentally relies on the reliability, consistency, completeness, and accuracy of the data it contains. The present study proposes to analyze and adapt existing data quality assessment frameworks to the specific context of property graphs, taking into account the constraints inherent to this model — notably the absence of a rigid schema, the multiplicity of data sources, and the diversity of data types. We formulate a set of quality dimensions and indicators relevant to this context, and discuss feasible assessment methods, thereby laying the groundwork for a data quality evaluation process tailored to graph databases.

1 - Introduction

This study aims to establish data quality criteria for graph databases. We consider here **Property Graphs** endowed with **labels**.

Definition 1.

A property graph is a tuple $G = (N, E, \rho, \lambda, \sigma)$ such that:

1. N is a finite set of nodes (also referred to as vertices).
2. E is a finite set of directed edges (referred to as edges when direction is not considered).
3. $\rho : E \rightarrow (N \times N)$ is a total function that maps each edge in E to a pair $(n_{\text{source}}, n_{\text{destination}})$. This node pair is non-commutative, since $(n_A, n_B) \in \rho(E)$ does not necessarily imply $(n_B, n_A) \in \rho(E)$.
4. $\lambda : (N \cup E) \rightarrow \text{SET}^+(L)$ is a partial function that assigns a set of labels from L to each node or edge (λ is a labeling function for nodes and edges).
5. $\sigma : (N \cup E) \times P \rightarrow \text{SET}^+(V)$ is a partial function that maps property values V to properties P of nodes and edges. We shall subsequently denote, for all $(o, p) \in (N \cup E) \times P$, the assigned values $\sigma(o, p) = \{v_1, \dots, v_n\}$ as $(o, p) = \vec{v}$.

2 - Data Quality of a Property Graph

2.1 - Completeness

Definition 2.1.0

Completeness measures the amount of missing data in a graph database [1].

2.1.1 - Component Existence

The existence of connected or strongly connected components is a method for verifying data completeness. Indeed, edges model a large portion of the relationships between objects and carry significant semantic meaning.

More intuitively, verifying the existence of components among sets of nodes and edges enables the expression of path (resp. walk) constraints. Naturally, adding constraints such as path lengths or the membership of a set of labels to these paths constitutes robust tools for capturing complex semantic meaning.

Definition 2.1.1

Let G_p be a graph pattern modeling a connected (resp. strongly connected) component, a set O such that $O \in \{N, E, N \cup E\}$, and $L_O \subseteq L$ a set of labels. Such that $\forall (G_p, O, L_O)$, we have $\forall o \in O$ such that $\lambda(o) = L_O$, $\exists$ at least one occurrence of G_p for o . See **Figure 1** in the appendix.

2.1.2 - Node Degree

Data completeness can also be expressed through node degree, thereby enabling the expression of cardinality constraints.

Definition 2.1.2

Let $L_O \subseteq L$ be a set of labels, and $D_s, D_e \subset \mathbb{R}^2$ be sets representing, respectively, the allowed out-degree and in-degree ranges. Such that $\forall n \in N$ such that $\lambda(n) = L_O$ satisfies $d^+(n) \in D_s$ and $d^-(n) \in D_e$. See **Figure 2** in the appendix.

2.2 - Validity

Definition 2.2.0

Validity measures the validity of data formats.

2.2.1 - String Format

Definition 2.2.1

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$, and **Regex** encoding the expected format. Such that $\forall o \in O$ we verify that $\forall v \in \sigma(o, X)$, $\text{match}(v, \text{Regex}) = \text{True}$. See **Figure 3** in the appendix.

2.2.2 - Date Format

Definition 2.2.2

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$, and **Date_{fnt}** encoding the expected date format. Such that $\forall o \in O$ we verify that $\forall v \in \sigma(o, X)$, $\text{match}(v, \text{Date}_{\text{fnt}}) = \text{True}$. See **Figure 4** in the appendix.

2.2.3 - Finite Data Domain

Definition 2.2.3

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$, I a data domain (including non-atomic values), and C an optional constraint (see Definition 2.3.2). Such that $\forall o \in O$ we verify $\text{SET}(\sigma(o, X)) \subseteq I \wedge C(o) = \text{True}$. See **Figure 5** in the appendix.

2.2.4 - Set-Based Labeling

Definition 2.2.4

Let $O \in \{N, E, N \cup E\}$, $L_X, L_Y \subseteq L$, and $\text{Op}_{\text{set}} \in \{\subset, \subseteq, \setminus\}$ a set operator. Such that $\forall o \in O$ such that $L_X \subseteq \lambda(o)$ satisfies $\lambda(o) \text{Op}_{\text{set}} L_Y = \text{True}$. See **Figure 6** in the appendix.

Note that only the partial implementation of this definition is meaningful within the **Neo4j** graph database framework, as edges (*Relationships*) may carry only a single label.

2.2.5 - Clustering-Based Labeling

The intuition is as follows: similar nodes should share the same label set. To measure labeling quality, we seek to group similar nodes in order to detect labeling errors. The approach described below is inspired by an embedding system motivated by [2]. The proposed approach proceeds as follows:

1. Define a similarity criterion between two nodes: we focus here on node labels and their semantic meaning. Our interest thus lies in the relationships between different label sets. These relationships are modeled by a semantically rich concept: edges. Indeed, edges are characterized by a directional node pair and a label set. We therefore propose to encode this semantic meaning as character strings. Thus the following edge:

$(\text{Node}_1: \{\text{Student}, \text{Person}\}) - [\text{Edge}: \{\text{EnrolledIn}\}] \rightarrow (\text{Node}_2: \{\text{University}\})$

Would be translated as “OU:EnrolledIn:University” (which we term a *Token*) from the perspective of Node_1 , and as “IN:EnrolledIn:StudentPerson” from that of Node_2 .

2. Define a method for computing similarity between two *Tokens*. Since a *Token* encodes complex semantic relationships as a character string, the use of edit distance seems most appropriate. We therefore employ **Levenshtein** similarity to compute the similarity between two *Tokens*.
3. Define a method for computing similarity between two nodes. We consider both their relationships and their labels, combining similarity scores along these two dimensions. We use the **Jaccard** index to compute node similarity with respect to label sets, such that $\forall n_1, n_2 \in N^2$, $\text{Similarity}_{\text{Labels}} = 1 - \frac{|\lambda(n_1) \cap \lambda(n_2)|}{|\lambda(n_1) \cup \lambda(n_2)|}$.

The similarity between two nodes with respect to their *Token* sets is computed using **Monge-Elkan** (ME) similarity, such that $\forall n_1, n_2 \in N^2$, $\text{ME}(n_1, n_2) = \frac{1}{|\sigma(n_1, \{\text{Tokens}\})|} \sum_{t_1 \in \sigma(n_1, \{\text{Tokens}\})} \max_{t_2 \in \sigma(n_2, \{\text{Tokens}\})} (\text{Levenshtein_Similarity}(t_1, t_2))$ (resp. $\text{ME}(n_2, n_1)$), such that $\text{Similarity}_{\text{Tokens}} = \frac{\text{ME}(n_1, n_2) + \text{ME}(n_2, n_1)}{2}$. Token similarity could alternatively be computed using the **Fuzzy Jaccard** algorithm (less precise) or the **Kuhn-Munkres** algorithm (optimal but with $O(|N|^3)$ complexity).

The following algorithms are defined:

The *Tokenization* algorithm generates a set of *Tokens* (representing the relationships of a node) for all nodes possessing at least one incoming or outgoing edge. Equivalently: any node with non-zero in- or out-degree acquires a “Tokens” property storing the set of *Tokens* generated for it.

The *CreateTokens* algorithm generates a set of connected nodes representing the distinct *Tokens* produced by the *Tokenization* algorithm. Once these nodes are created, *CreateTokens* computes the **Levenshtein** similarity between each pair of *Tokens* and stores it as an edge between them.

Tokenization(G PG.):

```

1 // Step - 1
2 for  $e$  in  $E$  do
3    $(n_s, n_d) \leftarrow \rho(e)$ 
4    $c_s \leftarrow \text{'OU:'} + \lambda(e) + \text{'.'} + \lambda(n_d)$ 
5    $c_d \leftarrow \text{'IN:'} + \lambda(e) + \text{'.'} + \lambda(n_s)$ 
6    $t_s \leftarrow \sigma(n_s, \{\text{Tokens}\})$ 
7    $t_d \leftarrow \sigma(n_d, \{\text{Tokens}\})$ 
8    $\sigma(n_s, \{\text{Tokens}\}) \leftarrow t_s \cup c_s$ 
9    $\sigma(n_d, \{\text{Tokens}\}) \leftarrow t_d \cup c_d$ 
10 end

```

The *Tokenization* algorithm has linear time complexity $O(n)$ (where $n = |E|$) and worst-case (very rare) space complexity $O(n \times m)$ (where $n = |N|$ and $m = |L|$). The *CreateTokens* algorithm has polynomial time complexity $O(n^2)$ (where $n = |N|$) and worst-case (very rare) space complexity $O(n^2)$ (where $n = 2|E|$).

CreateTokens(G PG.):

```

1 // Step - 2
2 vocab  $\leftarrow \emptyset$ 
3 for  $n$  in  $N$  do
4   tokens  $\leftarrow \sigma(n, \{\text{Tokens}\})$ 
5   if tokens  $\neq \text{NULL}$  do
6     vocab  $\leftarrow \text{vocab} \cup \text{tokens}$ 
7   for idx in  $[0; |\text{tokens}| - 1]$  do
8      $n \leftarrow \text{newNode}()$ 
9      $N \leftarrow N \cup \{n\}$ 
10     $\lambda(n) \leftarrow \{\text{TOKEN}\}$ 
11     $\sigma(n, \{\text{VAL}, \text{ID}\}) \leftarrow (\text{vocab}[\text{idx}], \text{idx})$ 
12  // We compute the similarity between the
  tokens.
13  for  $(n_1, n_2)$  in  $N^2$  do
14    if  $\lambda(n_1) = \lambda(n_2) = \{\text{TOKEN}\}$ 
15    and  $\sigma(n_1, \{\text{ID}\}) < \sigma(n_2, \{\text{ID}\})$  do
16       $t1, t2 \leftarrow \sigma(n_1, \{\text{VAL}\}), \sigma(n_2, \{\text{VAL}\})$ 
17      sim  $\leftarrow \text{Levenshtein\_Similarity}(t1, t2)$ 
18       $e \leftarrow \text{newEdge}()$ 
19       $E \leftarrow E \cup \{e\}$ 
20       $\rho(e) \leftarrow (n_1, n_2)$ 
21       $\lambda(e) \leftarrow \{\text{SIMILAR}\}$ 
22       $\sigma(e, \{\text{SCORE}\}) \leftarrow \text{sim}$ 
23 end

```

Once the *Tokenization* and *CreateTokens* algorithms have been applied, node groupings can be analyzed using Token set similarity alongside **Jaccard** similarity for label sets. Grouping is performed pairwise, storing only a single edge connecting nodes that should be “Merged” (those that should share a similar label set) or “Split” (those that should not share a similar label set). This selection is governed by two similarity thresholds: the first pertains to label similarity (used to filter node pairs that may exhibit labeling errors), and the second concerns Token similarity, determining whether a “Merge” / “Split” edge is created.

```

Merge( $G$  PG.,  $t_e$  label similarity threshold,  $t_t$  Token similarity threshold):
1 // Step - 3: Detection of nodes that should belong
2 // to the same label cluster.
3 for  $(n_1, n_2)$  in  $N^2$  do
4   if  $\sigma(n_1, \{ID\}) < \sigma(n_2, \{ID\})$ 
5   and  $\sigma(n_1, \{Tokens\}) \neq \text{NULL}$ 
6   and  $\sigma(n_2, \{Tokens\}) \neq \text{NULL}$  do
7     SimilarityLabels  $\leftarrow \frac{|\lambda(n_1) \cap \lambda(n_2)|}{|\lambda(n_1) \cup \lambda(n_2)|}$ 
8     if SimilarityLabels  $\geq t_e$  do
9       skip this iteration;
10    end
11
12    SimilarityTokens  $\leftarrow \frac{\text{ME}(n_1, n_2), \text{ME}(n_2, n_1)}{2}$ 
13    if SimilarityTokens  $\geq t_t$  do
14       $e \leftarrow \text{newEdge}()$ 
15       $E \leftarrow E \cup \{e\}$ 
16       $\rho(e) \leftarrow (n_1, n_2)$ 
17       $\lambda(e) \leftarrow \{\text{MERGE}\}$ 
18    end
19  end
20 end

```

The *Merge* algorithm described above therefore detects all node pairs whose label similarity is $<$ the threshold t_e , and for which Token similarity is $\geq$ the threshold t_t . In other words, the algorithm detects nodes that, based on their relational similarity (*Token*), should share a more similar label set (and hence should be merged). See **Figure 7** in the appendix.

This algorithm can readily be adapted to detect the inverse: “Split” designating node pairs that should not share similar label sets. To effect the necessary changes, it suffices to modify lines [8, 21, 25] of the *Merge* algorithm as follows.

```

Split( $G$  PG.,  $t_e, t_t$ ):
1 // [...]
2 // Line 8:
3 if SimilarityLabels  $\leq t_e$  do
4 // [...]
5 // Line 21:
6 if SimilarityTokens  $\leq t_t$ 
7   // [...]
8   // Line 25:
9    $\lambda(e) \leftarrow \{\text{SPLIT}\}$ 

```

2.3 - Consistency

Definition 2.3.0

Consistency measures the validity of relationships within the graph database.

2.3.1 - Functional Dependency (FD)

Definition 2.3.1

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, and $X, Y \subseteq P$; we define $(O, L_O, X \rightarrow Y)$ as a **FD**. Such that $\forall o_1, o_2 \in O^2$ such that $\lambda(o_1) = L_O$ and $\lambda(o_2) = L_O$ satisfies $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. See **Figure 8** in the appendix.

2.3.2 - Conditional Functional Dependency (CFD)

Definition 2.3.2.a

A **condition** is a tuple $C = (P_C, \text{VAL}, f, \text{NEXT})$ such that:

1. $P_C \subseteq P$ is the set of properties that must satisfy the condition.
2. $\text{VAL} \in \{\text{constant}, P\}$ is the comparison value. The “constant” may be of any data type (including non-atomic types).
3. $f : (N \cup E, P_C, \text{VAL}) \rightarrow \text{Boolean}$, is a function for verifying the condition on an object (e.g., “=”, “<”, “ $\in$ ”, etc.). We shall denote by $C(o)$ the fact that object o satisfies $f(o, P_C, \text{VAL})$, given P_C and VAL as defined in C .
4. $\text{NEXT} \in \{\emptyset, (\text{Condition}, \text{Boolean operator})\}$ is an optional second condition to be verified (enabling its combination with the first via the “Boolean operator”).

Definition 2.3.2.b

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, C a condition (see Definition 2.3.2), and $X, Y \subseteq P$; we define $(O, L_O, C, X \rightarrow Y)$ as a **CFD**. Such that $\forall o_1, o_2 \in O^2$ such that $\lambda(o_1) = L_O$ and $\lambda(o_2) = L_O$ and $C(o_1) = \text{True}$ and $C(o_2) = \text{True}$ satisfies $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. See **Figure 9** in the appendix.

2.3.3 - Graph Pattern Functional Dependency (GFD)

Definition 2.3.3

Let G_p be a graph pattern from which we derive $G'(N', E')$, a subgraph of G corresponding to G_p ; $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, and $X, Y \subseteq P$; we define $(O, L_O, G_p, X \rightarrow Y)$ as a **GFD**. Such that $\forall o_1, o_2 \in O^2$ such that $o_1, o_2 \in G'$ satisfies $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. See **Figure 10** in the appendix.

An alternative approach (fundamentally different) termed *gFD* [3] consists of excluding all nodes that do not possess all properties $(X \cup Y)$ defined by the *FD*.

2.3.4 - Query-Based Validation

Functional dependencies (*FD*, *CFD*, and *GFD*) are powerful tools. However, they are complex to extend so as to fully capture the semantic richness expressible by queries. For this reason, an additional validation approach — modeled after *dbt* — would consist of validating or invalidating queries written by the user.

Definition 2.3.4

This query-based validation system is defined by a tuple $dgt = (R, B)$ such that:

1. R is a query, as expressive as the query language permits, that may or may not return objects.
2. B is a boolean value indicating whether R must return objects in order to validate the constraint it defines.

See **Figure 11** in the appendix.

2.4 - Integrity

Definition 2.4.0

Integrity measures the structural validity of a graph database.

2.4.1 - Property Schema Validity

No *DDL* standard has yet emerged for graph databases. We therefore define three integrity constraints:

1. Property Uniqueness:

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, and $X \subseteq P$ such that $\forall o_1, o_2 \in O^2$ satisfies $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow o_1 = o_2$. See **Figure 12** in the appendix.

2. Property Existence:

Let $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, and $X \subseteq P$ such that $\forall o \in O$ satisfies $\text{NULL} \notin \sigma(o, X)$. See **Figure 13** in the appendix.

3. Property Value Types:

Let $t : (V) \rightarrow \text{SET}^+(T)$ be a total function assigning a set of types T to a set of values V , $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$, and $Y \subseteq T$ such that $\forall o \in O$ satisfies $(t \circ \sigma)(o, X) \subseteq Y$. See **Figure 14** in the appendix.

Note that these constraints can be expressed in **Cypher** (the query language of **Neo4j**).

2.4.2 - Index Validity

The intuition is as follows: missing values on indexed properties may signal degradation of the integrity of the graph database.

Definition 2.4.2

Let $i : (N \times E) \rightarrow \text{BITSET}$ denote indexed properties; $\forall o \in (N \cup E)$ we verify $\text{NULL} \notin i(o) \odot \sigma(o, P)$. See **Figure 15** in the appendix.

2.4.3 - Normal Form of a Property Graph

Algorithm:

```

PG_3FN( $G$  PG.,  $F$  FD set,  $L$  Label set):
1  $F_{\min} \leftarrow \text{get\_min\_cover}(F)$ 
2  $D \leftarrow \emptyset$ 
3  $\forall (X \rightarrow A) \text{ in } F_{\min}$ :
4    $F_x \leftarrow F_{\min} - \{(X \rightarrow A)\}$ 
5   if ( $F_x \models X \rightarrow A$ ):
6      $F_{\min} \leftarrow F_x$ 
7   else
8      $D \leftarrow D \cup \{(XA, F_{\min}.\text{projection}(XA))\}$ 
9  $F^+ \leftarrow \text{atomic\_closure}(F)$ 
10 LoopA  $\forall R_p \text{ in } F^+$ :
11    $\forall (R, F_r)$ 
12     if ( $(R \rightarrow RP) \in F^+$ ):
13       continue LoopA
14    $\text{key} \leftarrow \text{find\_key}(R_p, F^+)$ 
15    $D \leftarrow D \cup \{(\text{key}, F_{\min}.\text{projection}(\text{key}))\}$ 

```

```

get_min_cover( $F$  FD set):
1  $F_{\min} \leftarrow \text{min\_cover}(F)$ 
2  $\forall (X \rightarrow A) \text{ in } F_{\min}$ :
3    $F_x \leftarrow F_{\min} - \{(X \rightarrow A)\}$ 
4    $\forall (X \rightarrow B) \text{ in } F_x$ :
5     if ( $YB \subseteq XA$ )
6       and ( $XA \not\subseteq Y^+$ )
7       and ( $F_x \models X \rightarrow A$ ):
8          $F_{\min} \leftarrow F_x$ 
9 return  $F_{\min}$ 

```

The algorithm is formulated within the framework of **gFD** and **gUC** [4], which can readily be instantiated by **FD** (see Definition 2.3.1). Whereas **CFD*s** and **GFD*s** (graph pattern

functional dependencies) are not meaningful in a normalization context, as the algorithm would normalize to 3NF only a fragment of the database. See **Figure 16** in the appendix.

2.5 - Uniqueness

Definition 2.5.0

Uniqueness measures the redundancy of a graph database.

2.5.1 - Edge Duplicates

Definition 2.5.1

$\forall e_1, e_2 \in E^2$, e_1 and e_2 are duplicates if and only if: $\rho(e_1) = \rho(e_2)$, $\lambda(e_1) = \lambda(e_2)$, and $\sigma(e_1, P) = \sigma(e_2, P)$. See **Figure 17** in the appendix.

2.5.2 - Node Duplicates

Definition 2.5.2

$\forall n_1, n_2 \in N^2$, n_1 and n_2 are duplicates if and only if: $\lambda(n_1) = \lambda(n_2)$ and $\sigma(n_1, P) = \sigma(n_2, P)$. See **Figure 18** in the appendix.

This definition could be relaxed by also taking into account the edges incident to nodes, stipulating that nodes sharing more than a given threshold of common edges are considered duplicates.

3 - Profiling of a Property Graph

The purpose of profiling a graph database is to obtain a dashboard view of the data distribution. This section therefore collects indicators of interest for characterizing the data of a property graph. These indicators do not constitute data quality metrics per se, as their general-purpose nature cannot capture the domain-specific use cases intrinsic to a graph database.

3.1 - Completeness

3.1.1 - Weakly Connected Components

Definition 3.1.1

Detection of connected components in the graph using the **WCC** algorithm.

3.1.2 - Strongly Connected Components

Definition 3.1.2

Detection of strongly connected components in the graph using the **SCC** algorithm (paths only are considered here).

3.2 - Validity

3.2.1 - Detection of Inconsistent Property Types

Definition 3.2.1

$\forall p \in P$ we verify that $\forall o \in N$ (resp. E), such that $\sigma(o, \{p\}) \neq \text{NULL}$, $\exists$ a unique type t_x such that $(t \circ \sigma)(o, \{p\}) = t_x$ (see Definition 2.4.1). A concise dashboard listing properties for which the type is not unique is constructed from this detection.

3.3 - Integrity

3.3.1 - Node Property Distribution

Definition 3.3.1

Analysis of the distribution of properties defined for nodes, grouped by their label set.

3.3.2 - Node Property Distribution by Label

Definition 3.3.2

Analysis of the distribution of properties defined for nodes, grouped by each label individually attached to them.

3.3.3 - Edge Property Distribution

Definition 3.3.3

Analysis of the distribution of properties defined for edges, grouped by their label set.

Note that this restricted definition is equivalent to the per-label analysis under **Neo4j**, as edges (*Relationships*) carry only a single label.

3.4 - Labeling

3.4.1 - Anomaly Detection via Clustering

Definition 3.4.1

Using the **FastRP** algorithm, an *embedding* is generated for each node from its numerical properties (*features*) and the graph topology. These *embeddings* are then used to identify node clusters via the **KNN** algorithm. Once these clusters have been identified, results are filtered to retain those whose similarity meets or exceeds a given threshold. Finally, the labels of each node are compared against those of other nodes within the same cluster to detect potential

labeling errors.

It should be noted that this method is somewhat fragile, particularly because the *embeddings* may be largely composed of default values (*padding*), thereby introducing significant bias into similarity computations. Alternative approaches, such as community detection using the **Louvain** algorithm, would also be applicable for this profiling use case.

3.5 - Readability

3.5.1 - Node Degree Distribution

Definition 3.5.1

Analysis of the degree distribution (in-degree and out-degree) of nodes grouped by their label set.

3.5.2 - Multi-edge Detection

Definition 3.5.2

Detection of edges sharing the same source and destination nodes, thereby forming a multi-graph. A concise dashboard summarizing the label sets of the source and destination nodes, as well as the edge label sets, is constructed from this detection.

3.5.3 - Graph Eccentricity Analysis

Definition 3.5.3

Analysis of graph eccentricity: computation of the graph's radius and diameter. One can readily envision leveraging this information to analyze a graph modeling a network, for instance.

3.6 - Outliers

3.6.1 - Numerical Outlier Detection

Definition 3.6.1

Detection of numerical outliers in node and edge properties. This likewise enables data characterization and the detection, where applicable, of invalid values.

3.6.2 - Transitive Node Influence Analysis

Definition 3.6.2

The transitive influence of a node is determined by computing its *Eigenvector Centrality*, a measure from graph theory used to assess the importance of a node. This metric is computed by taking into account both the number of connections of a node and the importance of the nodes to which it is connected.

This analysis enables measurement of node influence and the detection, where applicable, of nodes whose influence is inconsistent with the modeled domain.

3.6.3 - Average Transitive Influence Analysis

Definition 3.6.3

Analysis of the average transitive influence across the nodes of the graph.

4 - Implementation — Neo4j

Neo4j is a graph database offering a flexible implementation of the property graph model. Nodes are referred to as “Nodes” and edges as “Relationships”. All concepts in **Neo4j** are consistent with the definition established in the introduction (see Definition 1), with the exception that “Relationships” may carry only a single label. The implementation of the data quality framework established in this study is publicly available at [5].

4.1 - Testing Methods

In order to evaluate the relevance of the data quality criteria we have defined, these criteria were tested on a variety of graph databases. These include graph databases of impeccable quality, such as those used in the official **Neo4j** documentation. For more chaotic, less synthetic data, we employed graph databases derived from the transformation of knowledge bases (such as the *YAGO3 Knowledge Base*) and artificially degraded their data.

We deliberately avoided graph databases derived from relational database transformations, in order to decouple our evaluation from the relational model. Furthermore, no in-depth testing was conducted on **RDF** databases such as *DBpedia*, which proved semantically insufficient due to its limited label and property diversity.

4.2 - Test Results

4.2.1 - Northwind

The *Northwind* database, prominently featured in the official **Neo4j** documentation, was used for evaluation. This dataset presents numerous advantages, including semantic richness — particularly in terms of labels — enabling the evaluation of some of the most complex indicators (e.g., Clustering-Based Labeling 2.2.5). The data were therefore degraded to simulate a dynamic, non-synthetic database.

These degradations include the deletion of 5% of edges, the corruption of 5% of date formats, the violation of 5% of constraints (**FD**, existence, type, and uniqueness), and the modification of the label set for 2% of nodes in each distinct label group.

Results for *seed* 42 were conclusive for the detection of missing edges via the approach of Definition 2.1.1, as well as for the indicators defined in: 2.2.2, 2.3.1, 2.3.2, 2.3.3, 2.3.4, 2.4.1. Clustering-Based Labeling also proved conclusive, although experimentation revealed certain limitations in its approach.

Indeed, for a graph database in which all nodes are associated with a single label, the label similarity filtering for the *Merge* and *Split* algorithms becomes polarized, restricting the possible values of $\text{Similarity}_{\text{Labels}}$ to 0.0 and 1.0. This reduces the flexibility of the indicator and may inevitably lead to false positives (i.e., the erroneous detection of errors). Nevertheless, this approach remains effective in the majority of cases for capturing complex semantic meaning and detecting labeling errors.

Figure 19 shows an execution of the *Merge* algorithm, in which each distinct node color corresponds to a distinct label set. It is noteworthy that labeling error detection manifests as nodes with high degree. Note that the graph shown is restricted to edges e such that $\lambda(e) = \{\text{Merge}\}$. A more detailed examination of **Figure 20** and **Figure 21** clearly demonstrates that nodes whose labeling was degraded are detected as highly similar to their true labeling, through the intact semantic dimension of their relationships — the edges. Although our indicator constructs the full graph without excluding any similarity relation, one could readily envision

an analytical solution based, for instance, on mean node degree or a clustering method to reduce the number of nodes to be analyzed.

Finally, **Figure 22** clearly illustrates the labeling error detected by the *Split* algorithm. The central node to which all others are connected exhibits a clear labeling error, as all nodes linked to it share the same label set. In other words, the nodes from which edges e such that $\lambda(e) = \{\text{Split}\}$ are outgoing are pairwise similar. In any case, the indicator we have defined constitutes a robust tool for analyzing the labeling quality of a graph database.

4.2.2 - YAGO3

The second database used to evaluate our data quality criteria is *Yago3* [6]. To provide a meaningful point of comparison, only a minimal set of degradations was applied. Specifically, the values of 1% of edges were degraded, and the dataset was restricted to the first 50000 lines (yielding 63563 nodes and 50000 edges). Furthermore, edge predicates were used as edge labels.

The test results were particularly conclusive: our indicators detected no data quality issues on the non-degraded portion of the dataset. Unsurprisingly, the data quality of this knowledge base is exemplary. **23**, **24**, and **25** present a small yet representative sample of the data. The data homogeneity is such that the *Merge* and *Split* algorithms of the Clustering-Based Labeling indicator detected no errors.

The profiling carried out via Edge Property Distribution 3.3.3 revealed the absence of a key property for a negligible fraction of edges ($\sim 0.40\%$), as illustrated in **Figure 26**. Finally, **Figure 27** illustrates the use of Query-Based Validation 2.3.4 to detect the previously degraded data.

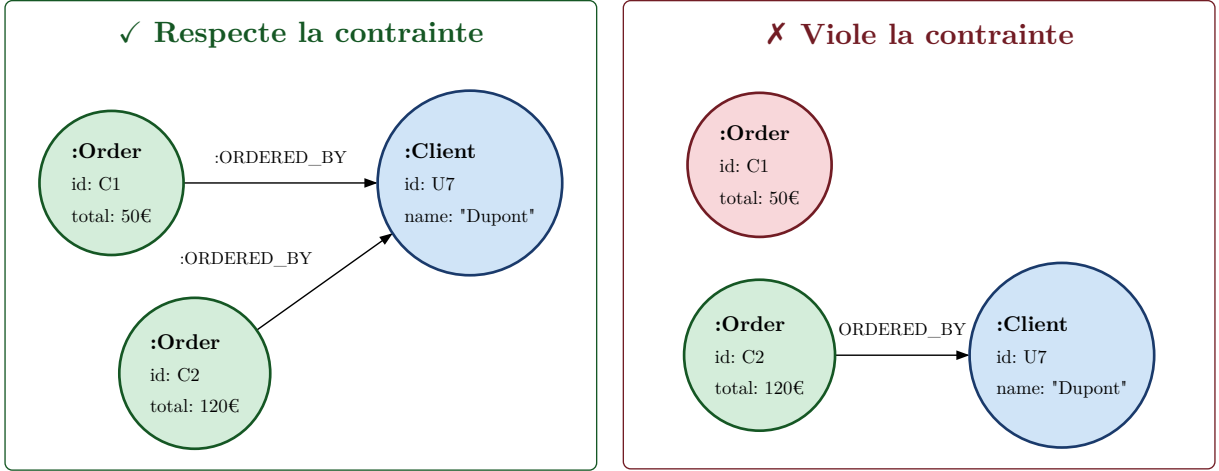
5 - Conclusion

This study has identified a substantial number of data quality indicators that prove both relevant and well-suited to the property graph model. Moreover, when combined with a profiling system, they provide a comprehensive overview of graph database content. The semi-structured nature of property graphs constitutes a powerful tool for expressing complex semantic concepts. Fully capturing the semantic richness of graph databases remains a significant challenge, given the diversity of their application contexts.

Significant challenges therefore remain, whether in the analysis of labeling quality, data validity (for which many complex checks could be standardized via a dedicated *DDL*), or the normal forms of a property graph.

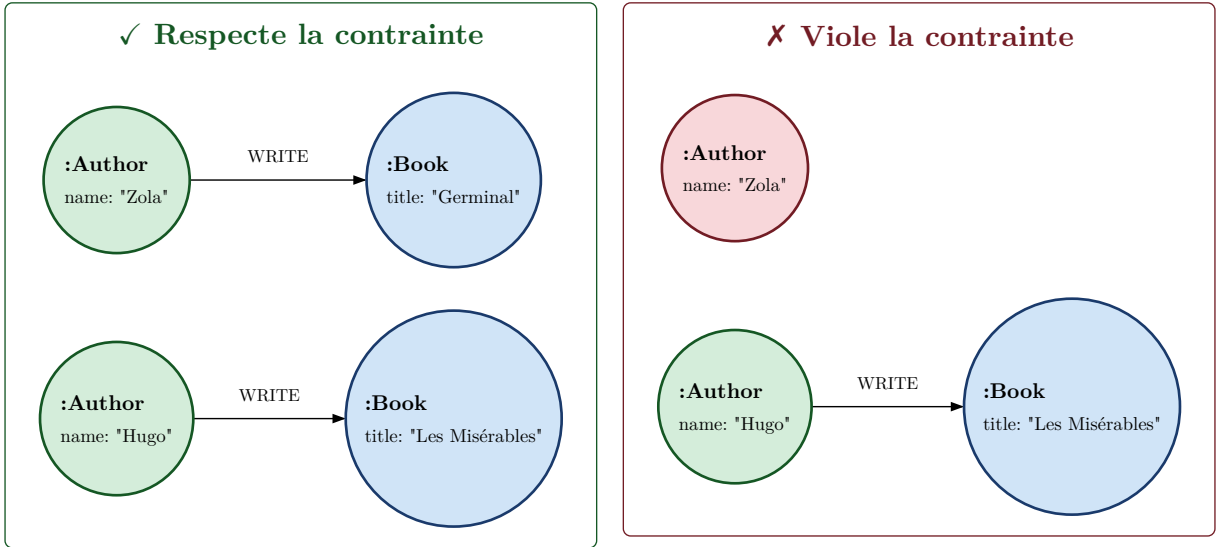
6 - Appendix

This appendix collects graph figures illustrating the definitions established in the preceding sections.



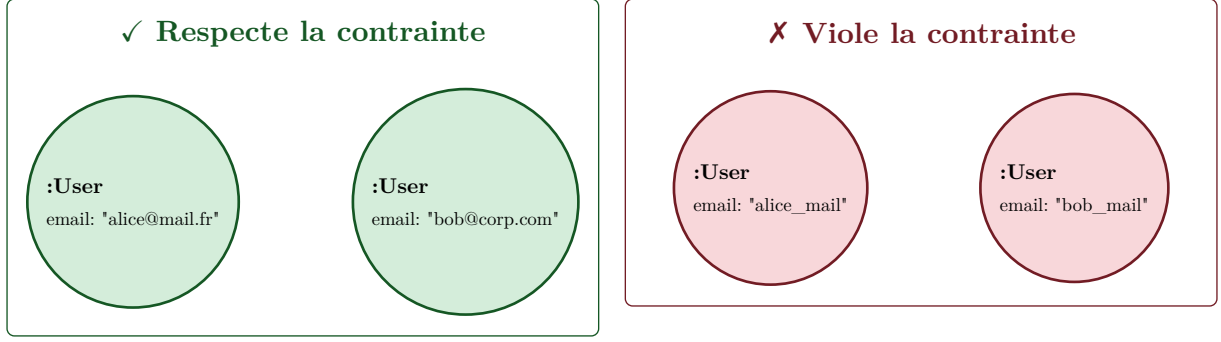
$$(G_p, O, L_O) = ((\{ : \text{Order} \}) - [\{ : \text{ORDERED_BY} : 1 \}] \rightarrow (\{ : \text{Client} \}), \{ N \}, \{ \text{Order} \})$$

Figure 1: Example for *Definition 2.1.1*



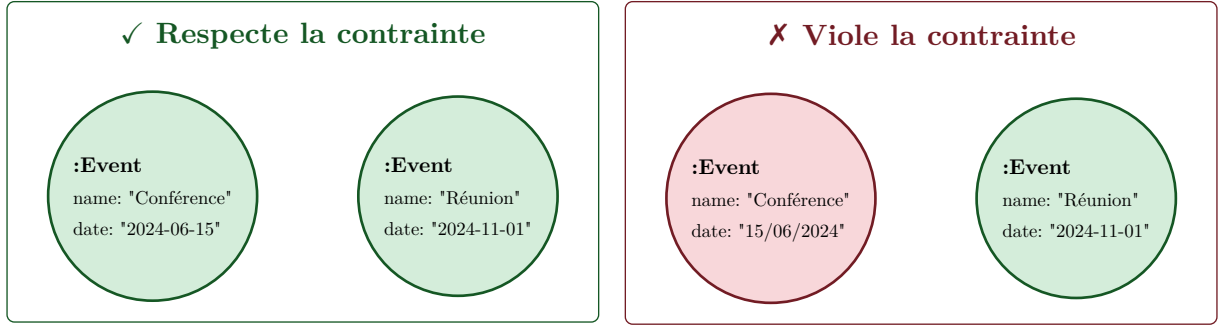
$$(L_O, D_s, D_e) = (\{ \text{Author} \}, \{ 1, 2, 3 \}, \{ 0 \})$$

Figure 2: Example for *Definition 2.1.2*



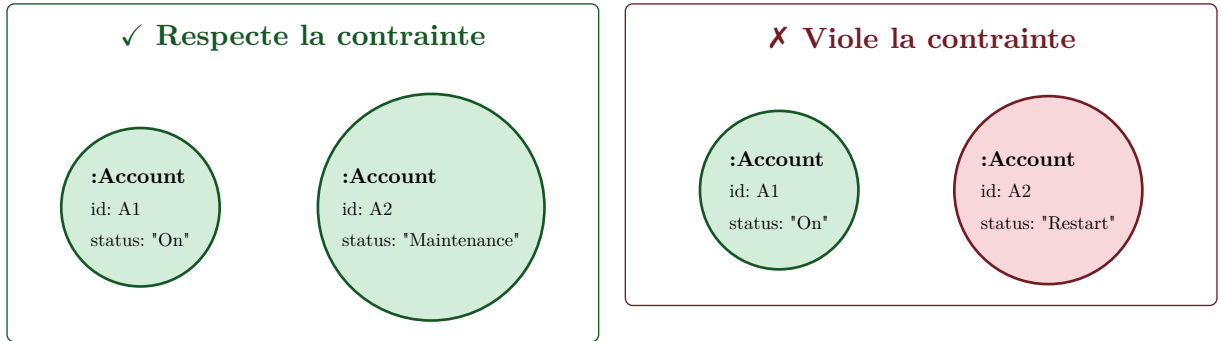
$$(O, L_O, X, \text{Regex}) = (N, \{\text{User}\}, \{\text{email}\}, "/^{\wedge}[\backslash w.]+\backslash @[\backslash w.]+\backslash .[a-z]\{2,\}")$$

Figure 3: Example for *Definition 2.2.1*



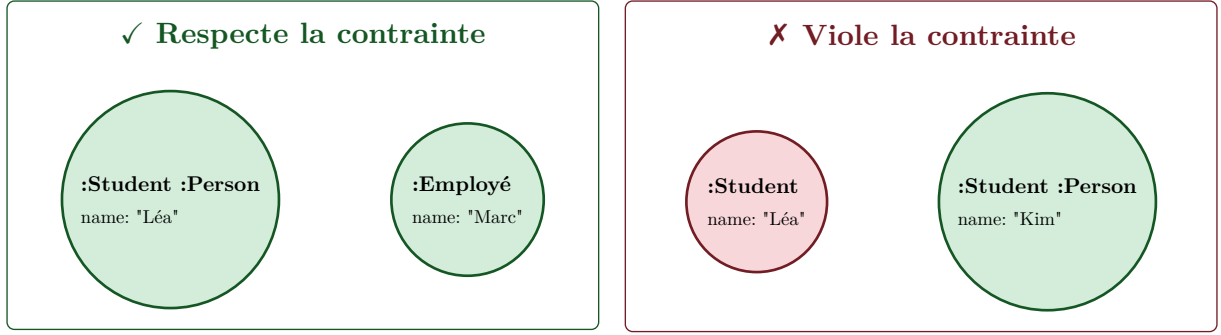
$$(O, L_O, X, \text{Date}_{\text{fmt}}) = (N, \{\text{Event}\}, \{\text{date}\}, "YYYY-MM-DD")$$

Figure 4: Example for *Definition 2.2.2*



$$(O, L_O, X, I) = (N, \{\text{Account}\}, \{\text{status}\}, \{\text{On}, \text{Off}, \text{Maintenance}\})$$

Figure 5: Example for *Definition 2.2.3*



$(O, L_X, L_Y, \text{Op}_{\text{ens}}) = (N, \{\text{Student}\}, \{\text{Person}\}, \subset)$
Figure 6: Example for Definition 2.2.4

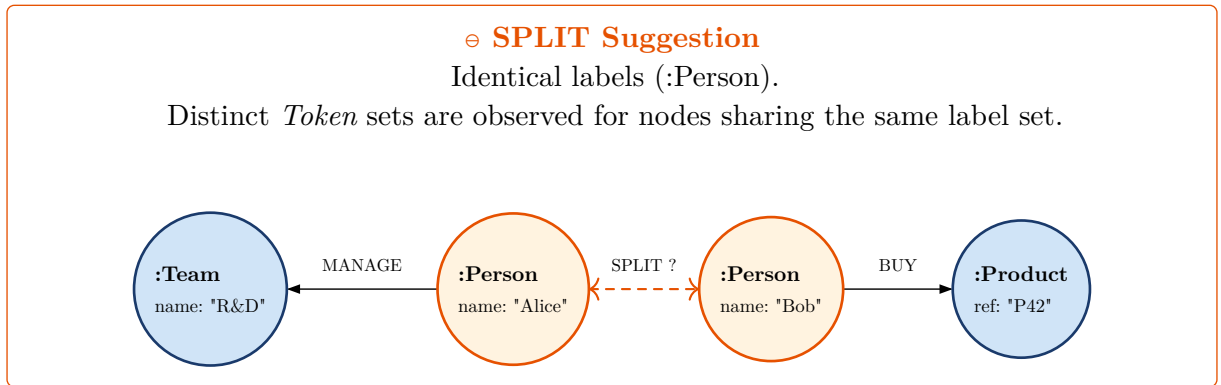
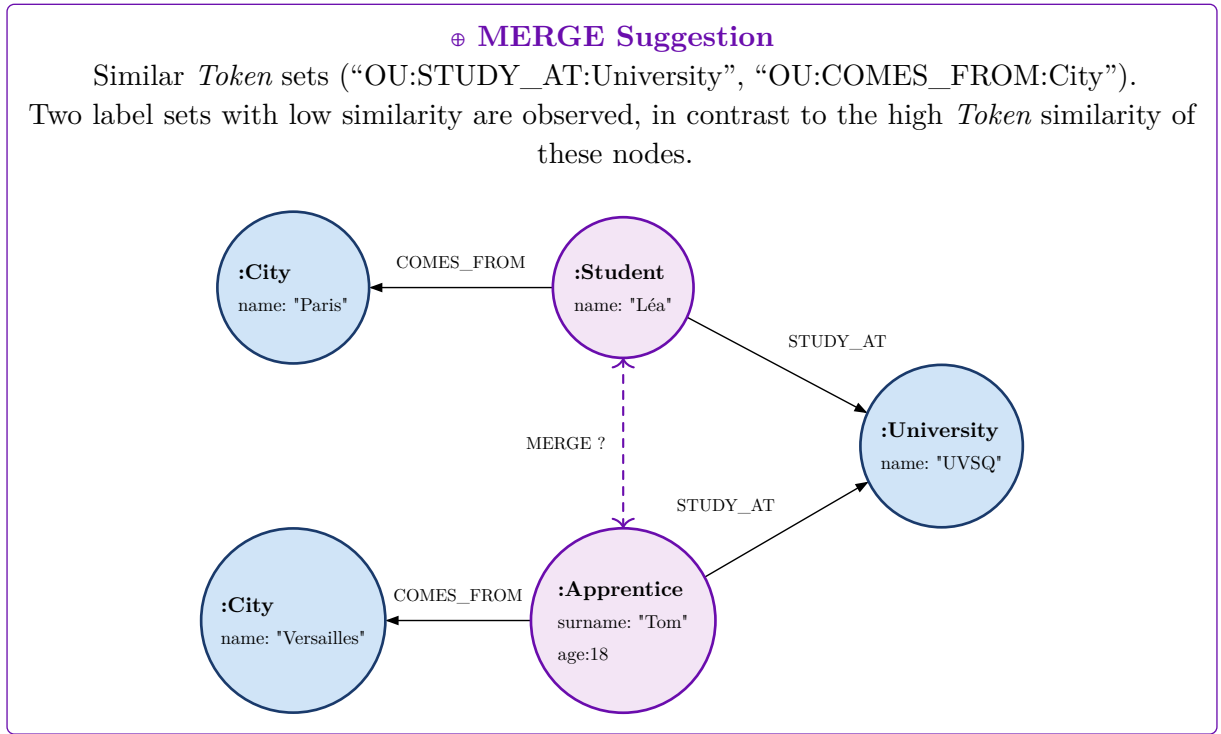
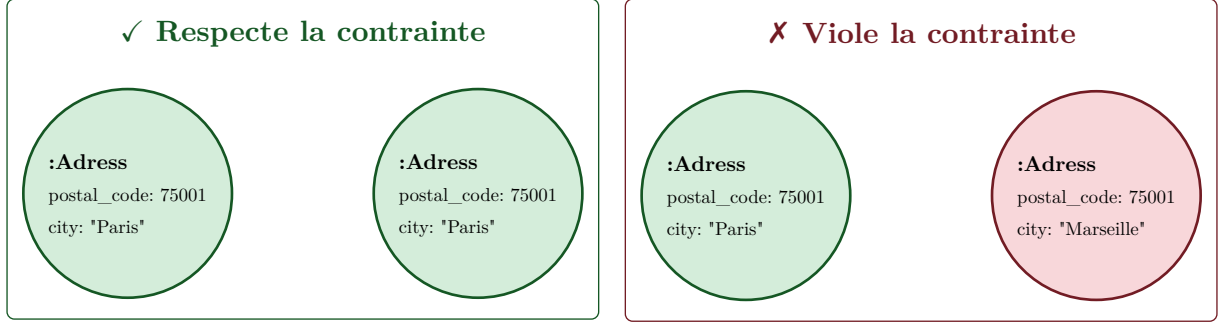
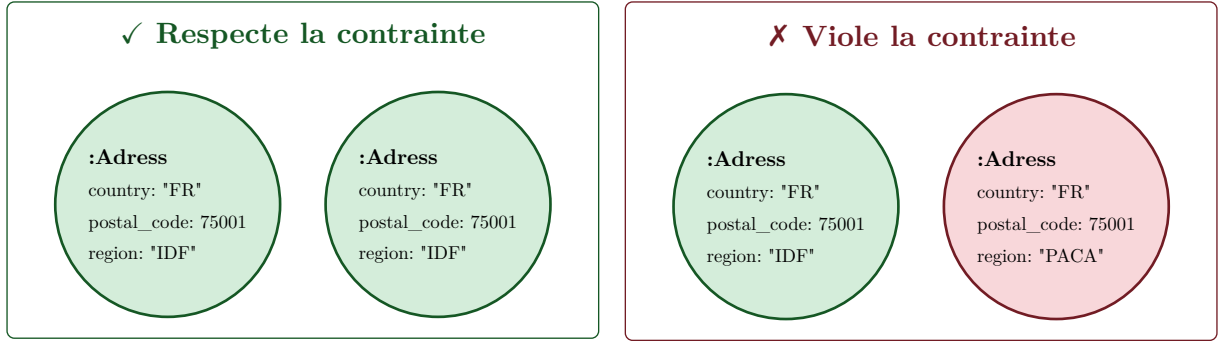


Figure 7: Example for Definition 2.2.5



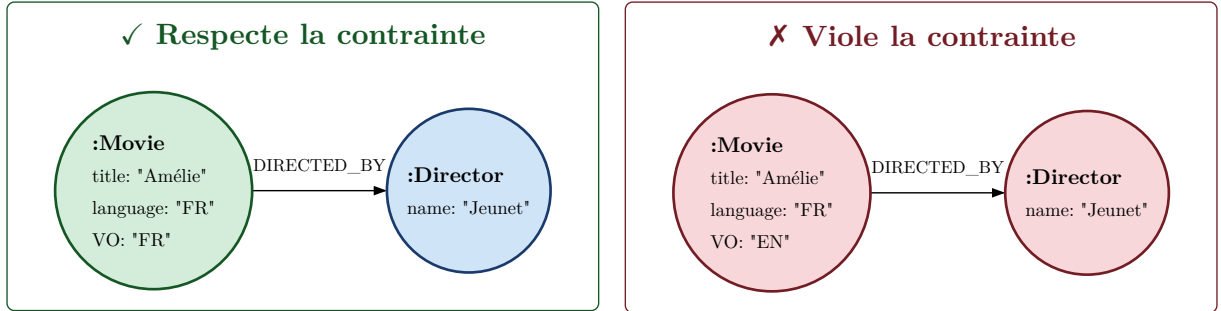
$$(O, L_O, X \rightarrow Y) = (N, \{\text{Adress}\}, \{\text{postal_code}\} \rightarrow \{\text{city}\})$$

Figure 8: Example for *Definition 2.3.1*



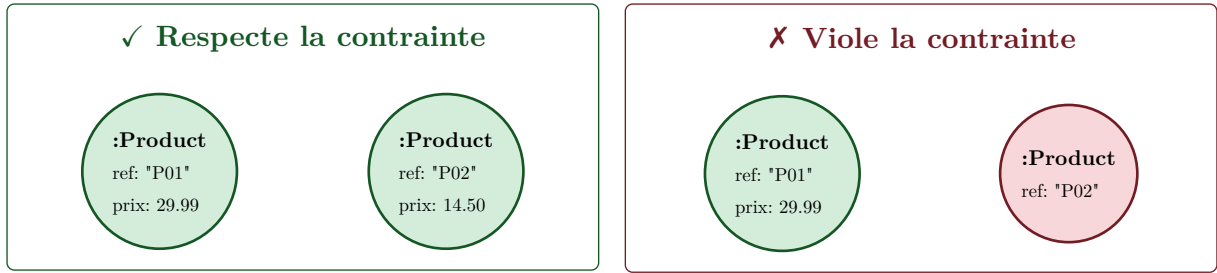
$$(O, L_O, C, X \rightarrow Y) = (N, \{\text{Adress}\}, (\{\text{country}\}, \text{"FR"}, =, \emptyset), \{\text{postal_code}\} \rightarrow \{\text{region}\})$$

Figure 9: Example for *Definition 2.3.2*



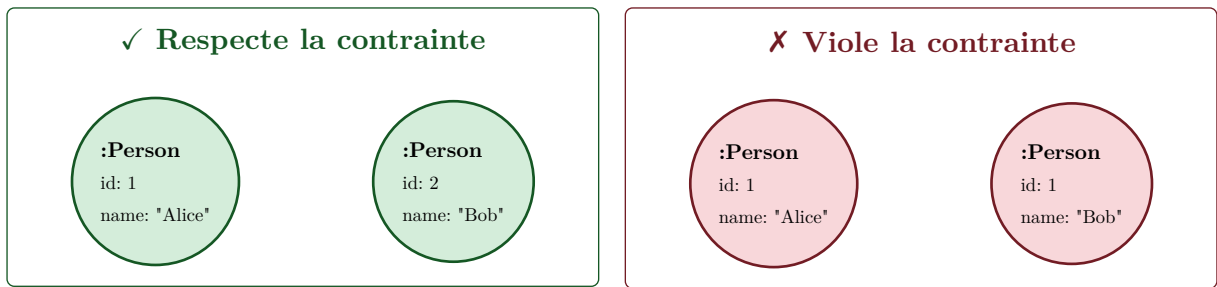
$$(G_p, O, L_O, X \rightarrow Y) = ((\{ : \text{Movie} \}) - [\{ : \text{DIRECTED_BY} \} : 1] \rightarrow (\{ : \text{Director} \}), \{N\}, \{\text{language}\} \rightarrow \{\text{VO}\})$$

Figure 10: Example for *Definition 2.3.3*



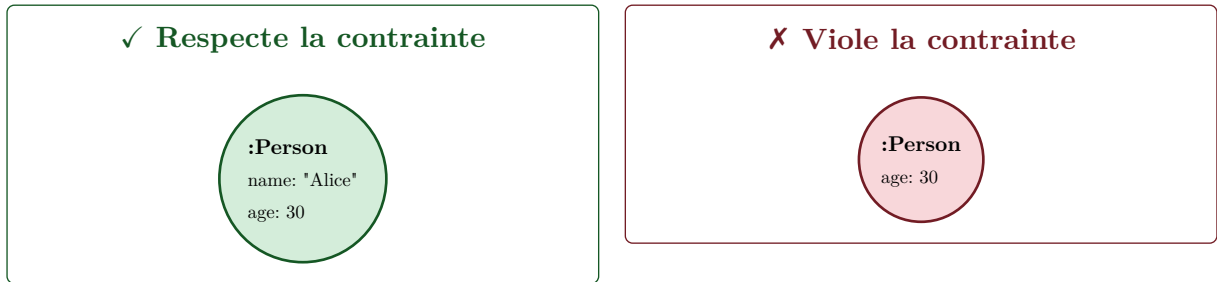
$(R, B) = (\text{"MATCH (n:Product) WHERE n.prix IS NULL"}, \text{False})$

Figure 11: Example for *Definition 2.3.4*



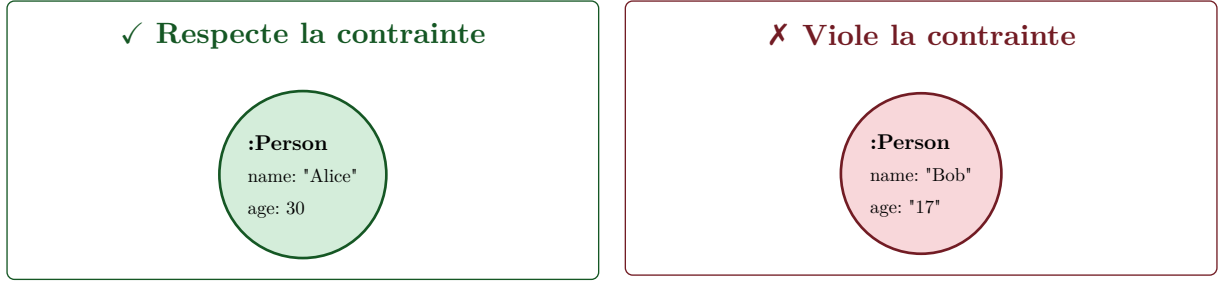
$(O, L_O, X) = (N, \{\text{Person}\}, \{\text{id}\})$

Figure 12: Example for *Definition 1 of 2.4.1*



$(O, L_O, X) = (N, \{\text{Person}\}, \{\text{name}\})$

Figure 13: Example for *Definition 2 of 2.4.1*



$(O, L_O, X, Y) = (N, \{\text{Person}\}, \{\text{age}\}, \{\text{Integer}\})$
Figure 14: Example for *Definition 3 of 2.4.1*



$(O, L_O, \text{Index}, X) = (N, \{\text{User}\}, \{\text{email}\}, \{\text{email}\})$
Figure 15: Example for *Definition 2.4.2*

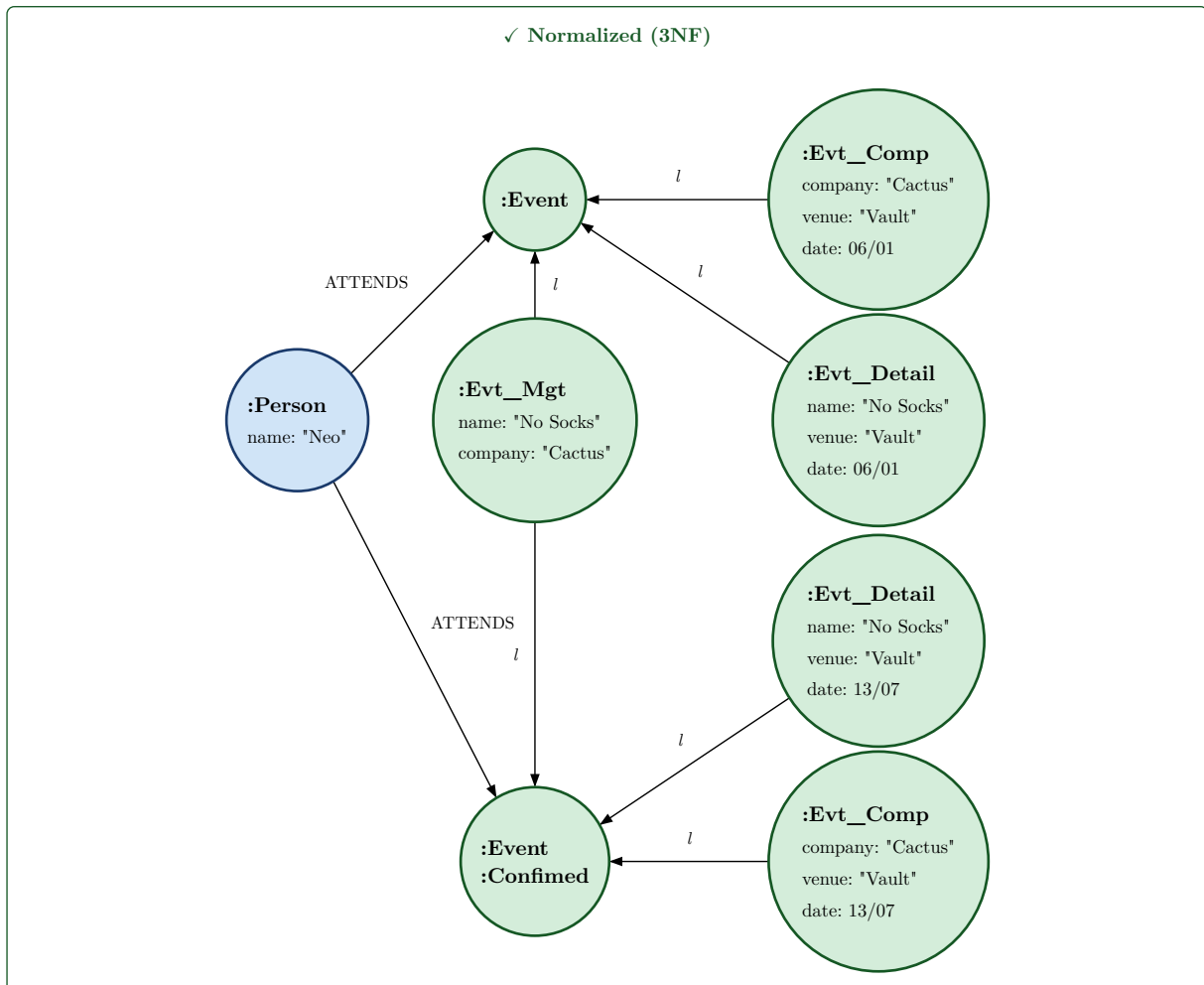
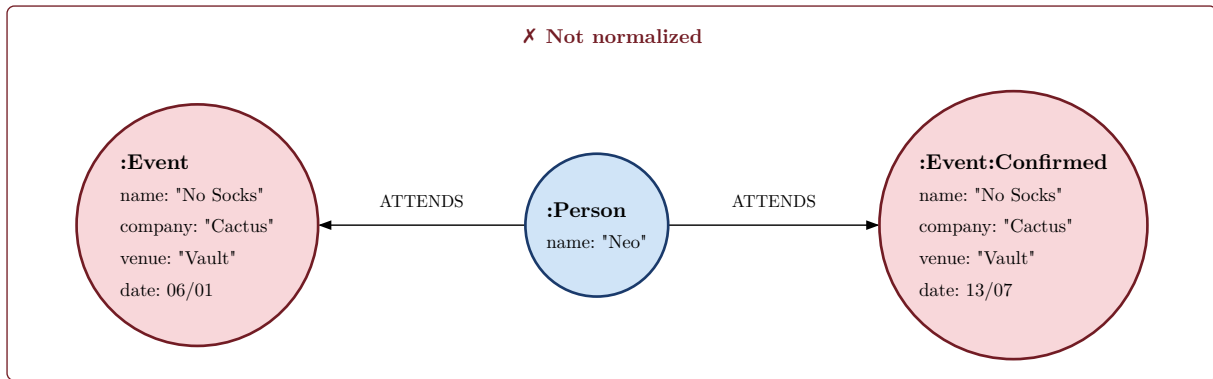


Figure 16: Example for *Definition 2.4.3*

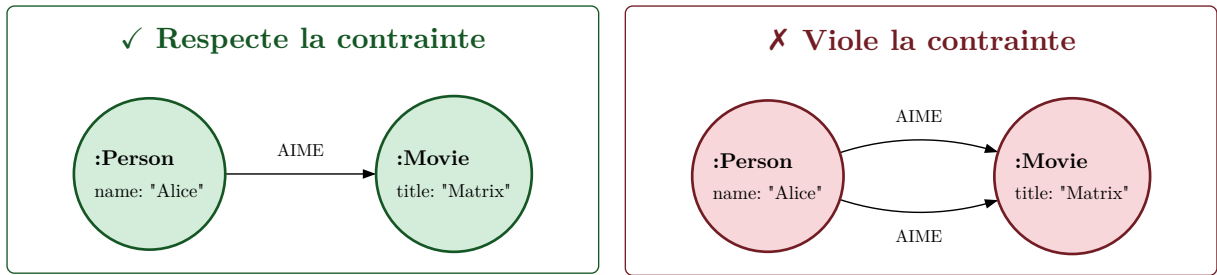


Figure 17: Example for *Definition 2.5.1*

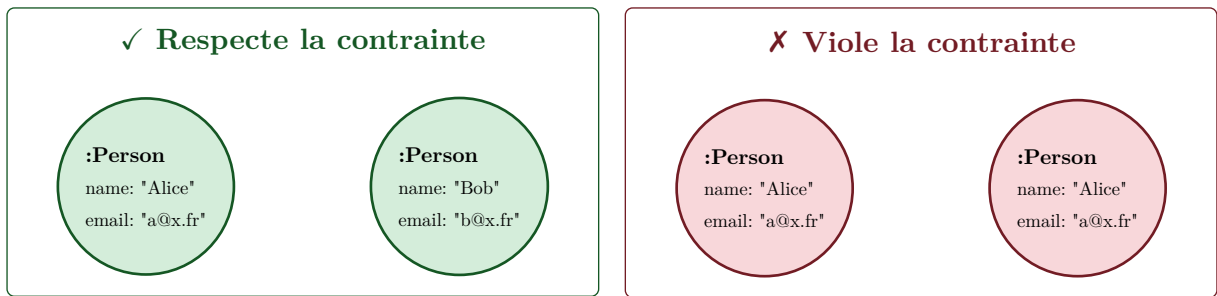
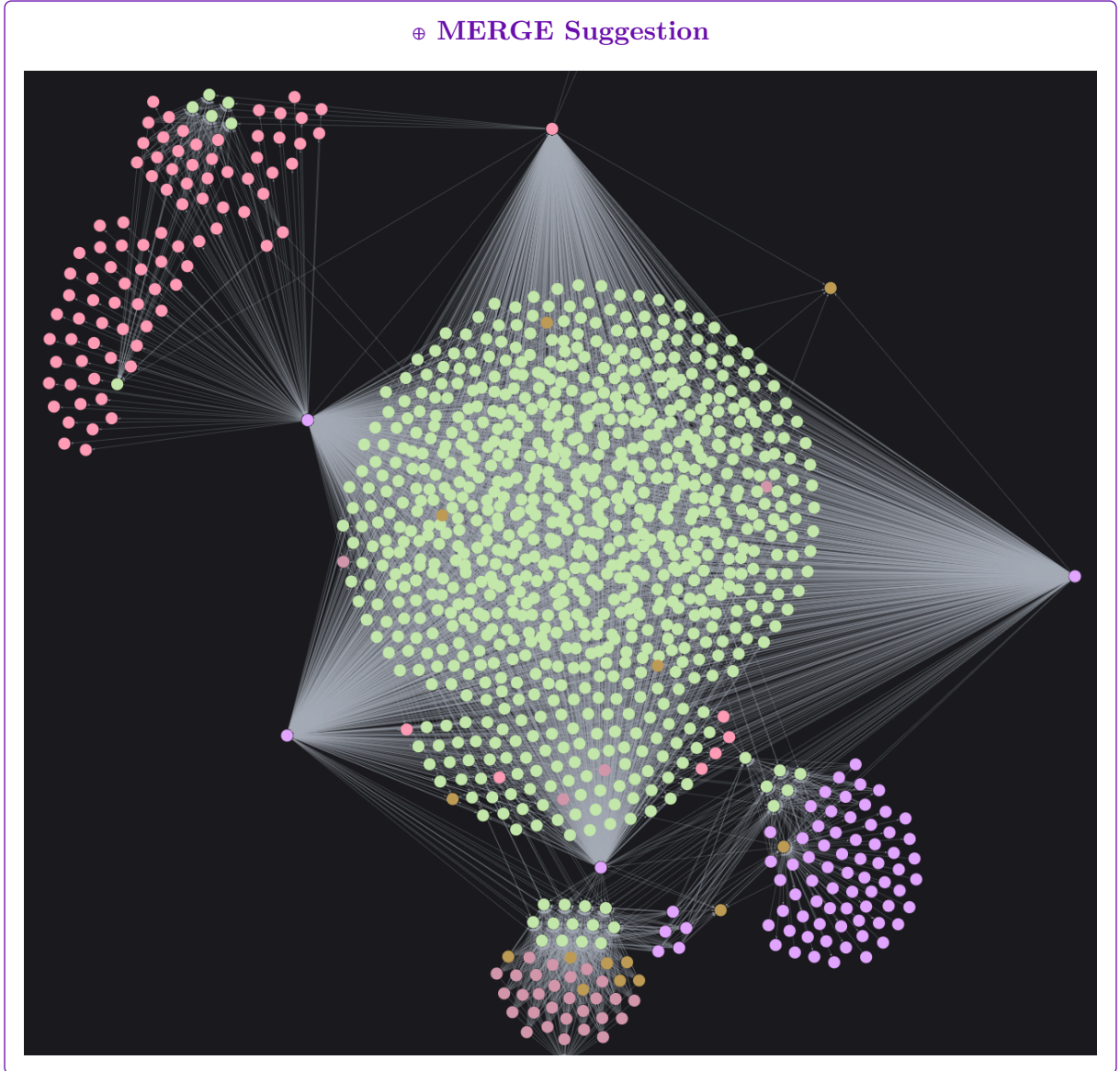


Figure 18: Example for *Definition 2.5.2*



Northwind database (see 4.2.1) degraded with *seed* 42 and
Merge executed with arguments $(t_e, t_t) = (0.4, 0.6)$.

Figure 19: Execution of the *Merge* algorithm on the *Northwind* database

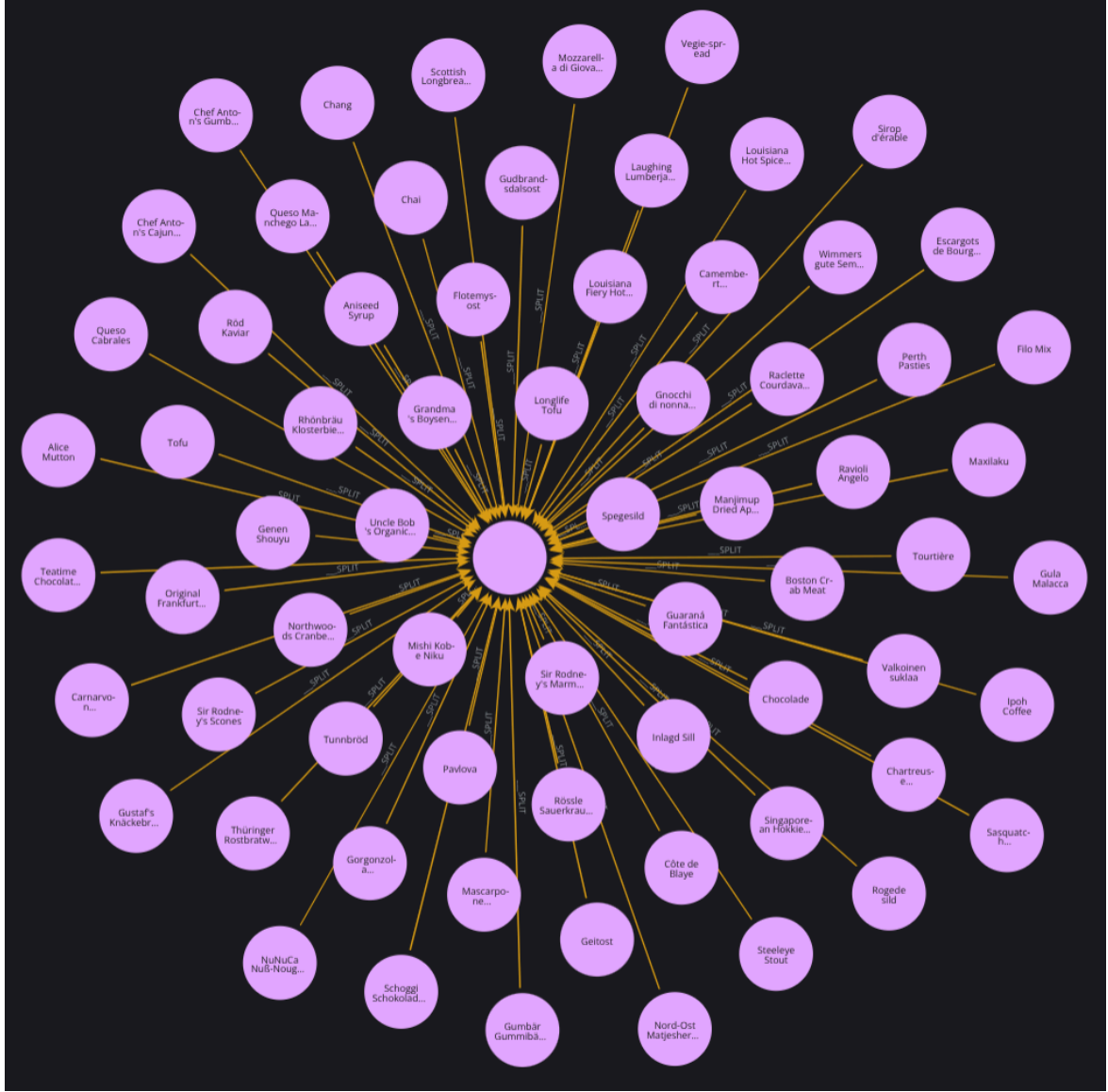
⊕ MERGE Suggestion

Figure 20: Execution of the *Merge* algorithm on the *Northwind* database

⊕ MERGE Suggestion

Figure 21: Execution of the *Merge* algorithm on the *Northwind* database

◉ SPLIT Suggestion



Northwind database (see 4.2.1) degraded with *seed* 42 and
Split executed with arguments $(t_e, t_t) = (0.7, 0.4)$.

Figure 22: Execution of the *Split* algorithm on the *Northwind* database

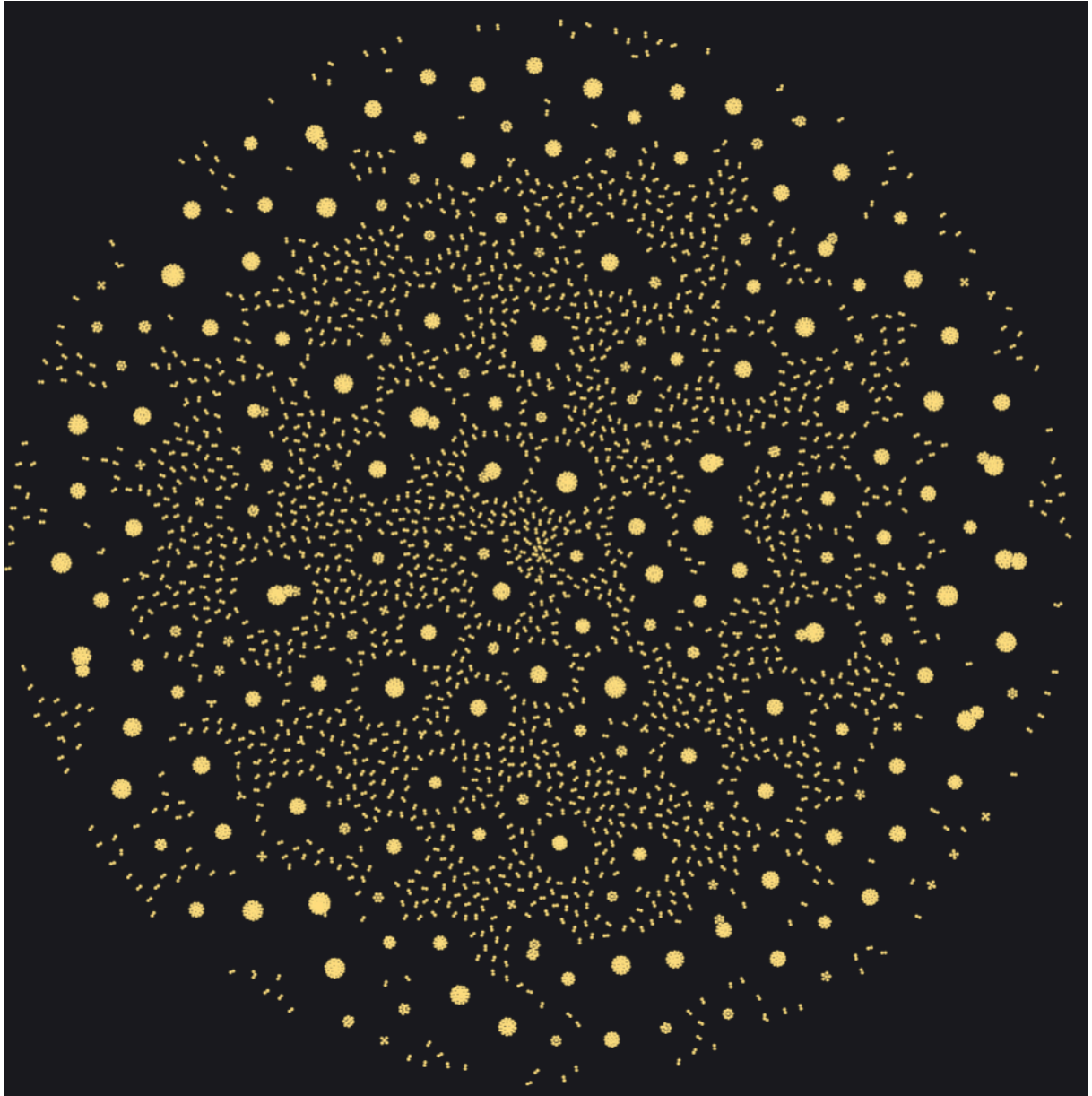
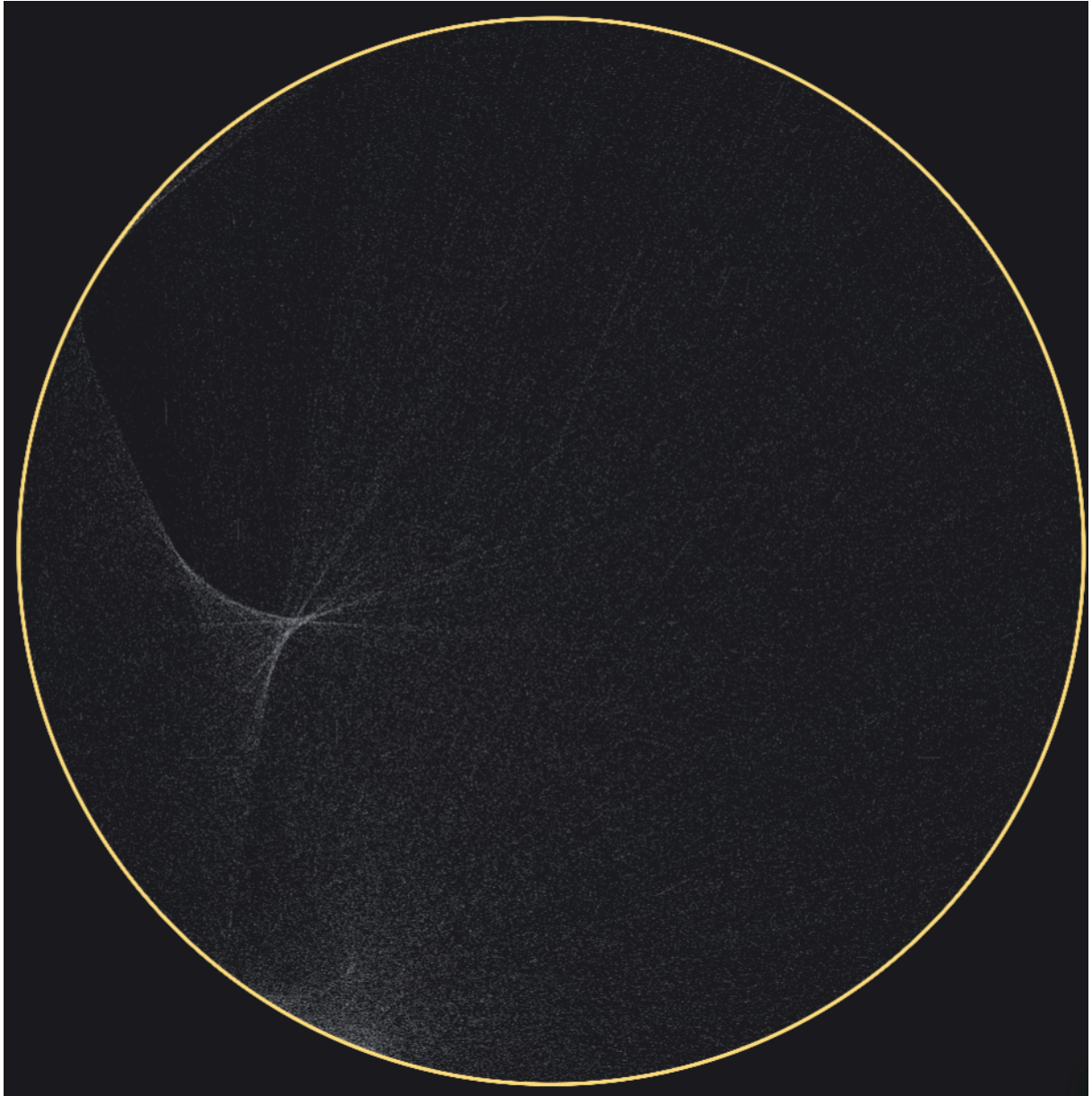


Figure 23: Sample from the *YAGO3* 4.2.2 database



White segments represent edges, and nodes form the outer circle.

Figure 24: Sample from the *YAGO3* 4.2.2 database

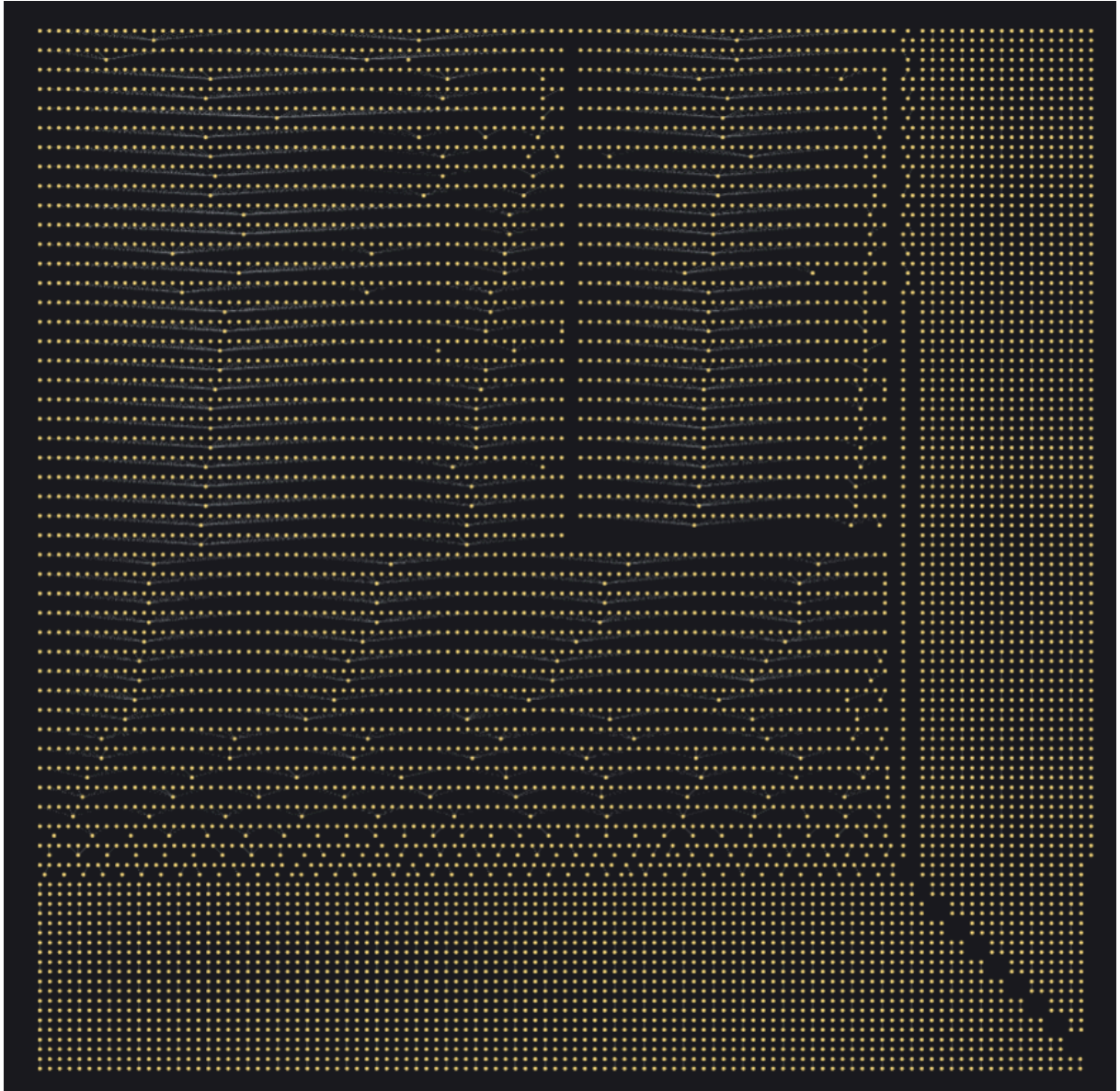


Figure 25: Sample from the *YAGO3* 4.2.2 database

	entity	label	count	properties_names	properties_count	properties_percent
0	Entity.RELATIONSHIP	<hasGloss>	1		1	100
1	Entity.RELATIONSHIP	<wasCreatedOnDate>	16349	date_value	16349	100
2	Entity.RELATIONSHIP	<diedOnDate>	10052	date_value	10046	99.9403
3	Entity.RELATIONSHIP	<diedOnDate>	10052		6	0.0597
4	Entity.RELATIONSHIP	<wasBornOnDate>	20850	date_value	20847	99.9856
5	Entity.RELATIONSHIP	<wasBornOnDate>	20850		3	0.0144
6	Entity.RELATIONSHIP	<wasDestroyedOnDate>	1231	date_value	1231	100
7	Entity.RELATIONSHIP	<happenedOnDate>	1517	date_value	1512	99.6704
8	Entity.RELATIONSHIP	<happenedOnDate>	1517		5	0.3296

Screenshot of the **data-quality** user interface [5].

Figure 26: Edge property profiling of the *YAGO3* 4.2.2 database

CDVQ - Data validation query

Validate your data with a query approach for complex purpose.

	Query	Should be empty
	match ()-[r]-() where r.date_value = "3000-01-01" return r	☒

Analyse

Result of the analysis:

	query	should_be_empty	found
0	match ()-[r]-() where r.date_value = "3000-01-01" return r	☒	926

Screenshot of the **data-quality** user interface [5].

Figure 27: Query-Based Validation 2.3.4 of the *YAGO3* 4.2.2 database

Bibliography

- [1] L. Cai and Y. Zhu, “The Challenges of Data Quality and Data Quality Assessment in the Big Data Era,” *Data Science Journal*, vol. 14, pp. Art.2, 1–10, 2016, doi: 10.5334/dsj-2015-002.
- [2] R. Giot, R. Bourqui, N. Journet, and A. Vialard, “Visual Graph Analysis for Quality Assessment of Manually Labelled Documents Image Database,” in *13th International Conference on Document Analysis and Recognition (ICDAR)*, Tunis, Tunisia, 2015. [Online]. Available: <https://hal.science/hal-01170011>
- [3] M. Manouvrier and K. Belhajjame, “PG-FD: Mapping Functional Dependencies to the Future Property Graph Schema Standard,” in *Proceedings of the 24th European Conference on Advances in Databases and Information Systems (ADBIS)*, 2024, pp. 45–59. doi: 10.1007/978-3-031-70626-4_4.
- [4] P. Skavantzios and S. Link, “Normalizing Property Graphs,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 16, no. 11, pp. 3031–3043, 2023, doi: 10.14778/3611479.3611506.
- [5] L. Desmarès, “data-quality.” [Online]. Available: <https://github.com/LugolBis/data-quality>
- [6] F. Mahdisoltani, J. Biega, and F. Suchanek, “Yago3: A knowledge base from multilingual wikipedias,” in *7th biennial conference on innovative data systems research*, 2014. [Online]. Available: <https://yago-knowledge.org/downloads/yago-3>